E-Commerce Sales Analytics and Product Recommendation using Hadoop, Hive and Spark

BCSE402L - Big Data Analytics (TH) | DA-3: Completion & Presentation

Team Members: K PRADEEP KUMAR - 23BLC1240, VAHRAN P - 23BLC1221, ASHVATH KUMAR - 23BLC1269, K.P ABHISHEK - 23BLC1292

Submission Date: 20 October 2026 | Dataset: Brazilian E-Commerce Public Dataset by Olist

Abstract

This project builds an end-to-end batch Big Data analytics pipeline over the Brazilian Olist e-commerce dataset. Nine relational CSVs (~1.55 M rows, 122 MB) are ingested into HDFS, cleaned and aggregated with MapReduce, modelled in Hive, cross-validated in Spark, and served from MongoDB. The pipeline answers business questions around monthly GMV, category and state revenue, payment preferences, delivery performance and customer repeat behaviour. A popularity + category-affinity recommender replaces an earlier ALS experiment that failed because of extreme sparsity, achieving a HitRate@10 of 0.0952 on a leave-one-out evaluation. A Streamlit dashboard provides interactive visualisation over the MongoDB serving store.

1. Problem Statement

Small and mid-size e-commerce marketplaces need affordable analytics to understand sales trends, geographic demand, payment behaviour and delivery quality. They also need product recommendations, but most public datasets are too sparse for collaborative filtering to work out of the box.

The problem addressed in this project is: **design and implement a batch Big Data pipeline that cleans, warehouses, analyses and serves e-commerce transaction data, and produce honest, evaluated product recommendations at a scale where single-machine tools become limiting.**

Why it matters: transaction data has high volume, variety and veracity. The Olist dataset spans orders, items, payments, reviews, products, sellers, customers and geolocation. Joining and aggregating these tables exercises the core Big Data concepts taught in BCSE402L.

2. Methodology

2.1 Software stack

Component	Version	Role
Apache Hadoop (HDFS + YARN)	3.5.0	Distributed storage and resource management, pseudo-distributed
Apache Hive	4.2.1	SQL warehouse with external and partitioned Parquet tables
Apache Spark / PySpark	4.2.0	DataFrame ETL, Spark SQL analytics, MLlib ALS experiment
MongoDB Community	8.3.11	Document serving store for recommendations and aggregates
OpenJDK	21	Single JDK shared by Hadoop, Hive and Spark
Python	3.11	PySpark driver, MongoDB loader and dashboard
Streamlit	1.50.0	Interactive dashboard

2.2 Dataset

File	Rows	Use
olist_orders_dataset	99,441	Order header, status and timestamps
olist_order_items_dataset	112,650	Line items: price, freight, product and seller
olist_customers_dataset	99,441	Customer IDs, unique IDs, city and state
olist_products_dataset	32,951	Category and physical attributes
olist_order_payments_dataset	103,886	Payment type, value and installments
olist_order_reviews_dataset	99,224	Review score and comment text
olist_sellers_dataset	3,095	Seller location
product_category_name_translation	71	Portuguese to English category names
olist_geolocation_dataset	1,000,163	Zip-code lat/long (ingested, not queried)

2.3 Pipeline workflow

1. **Ingestion.** Nine CSVs are uploaded to HDFS /olist/raw. A clean layer /olist/clean removes headers and normalises embedded newlines/quotes.
2. **MapReduce.** Three jobs compute revenue per product, orders per month and payment-type distribution. Java jobs use combiners and a partitioner; the streaming job uses Python mapper/combiner/reducer scripts.
3. **Hive.** Eight external tables and one Parquet table partitioned by purchase month support ten HQL analytics queries.
4. **Spark.** A curated Parquet fact table joins six tables. Six Spark SQL analyses mirror the Hive questions for cross-engine validation.
5. **Recommender.** An ALS experiment showed the matrix is too sparse. A popularity + category-affinity baseline is trained and evaluated with ranking metrics.
6. **MongoDB.** Aggregates and recommendations are loaded as documents and queried with NoSQL operations.
7. **Dashboard.** A Streamlit application reads from MongoDB and visualises KPIs, trends, geography, delivery and recommendations.

2.4 Architecture

The architecture is a classic lambda-style batch pipeline. Raw CSVs land in HDFS, are normalised, aggregated by MapReduce, modelled in Hive, enriched in Spark and served from MongoDB. This separation lets each tool do what it does best: HDFS for cheap durable storage, MapReduce for scalable aggregation, Hive for SQL analytics, Spark for in-memory joins and ML, and MongoDB for low-latency serving.

Fig. 1: Implemented end-to-end architecture

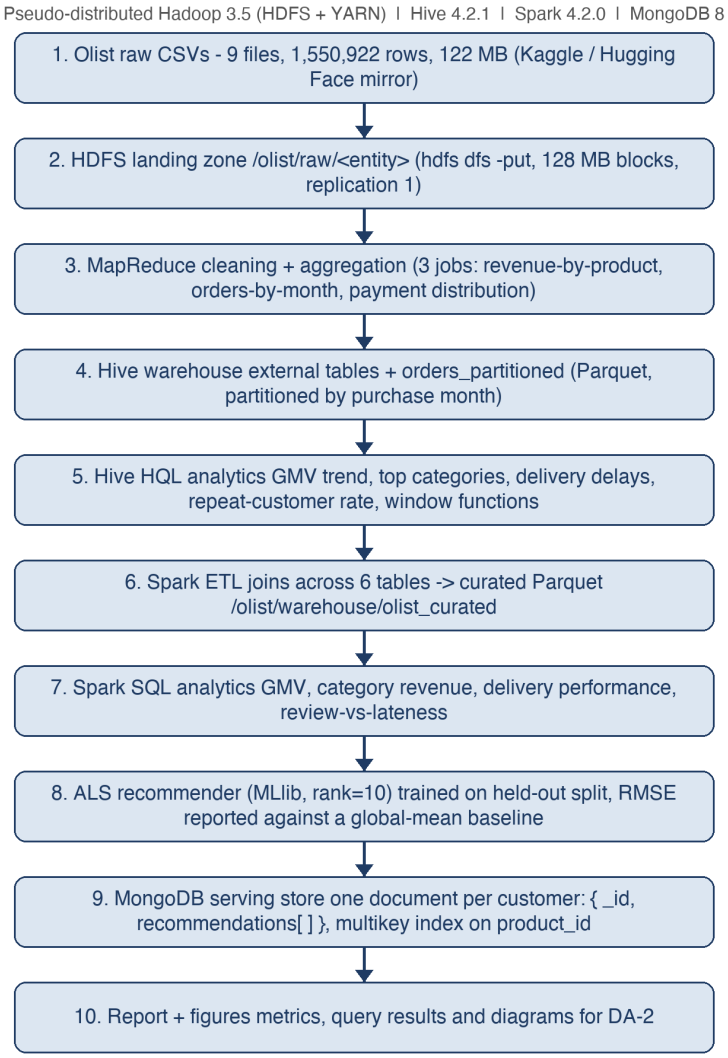


Fig. A: End-to-end implemented architecture.

Fig. 3: Project progress against the DA-1 timeline

Phase	Planned	Status and evidence
Phase 1 Setup + dataset	4 - 10 Aug	Hadoop 3.5 pseudo-distributed, Hive 4.2.1, Spark 4.2.0, MongoDB; dataset downloaded (9 CSVs, 122 MB)
Phase 2 Ingestion + cleaning	11 - 24 Aug	HDFS landing zone /olist/raw with one directory per entity; cleaning rules embedded in the MapReduce mappers
Phase 3 MapReduce + Hive	25 Aug - 7 Sep	3 MapReduce jobs (2 Java, 1 Streaming); Hive warehouse with 8 external tables + 1 partitioned Parquet table; 10 HQL analytics queries
Phase 4 Spark ETL + ALS	8 - 19 Sep	Spark ETL joining 6 tables to curated Parquet; 6 Spark SQL analyses; ALS recommender trained and evaluated on a held-out split
Phase 5 MongoDB + dashboard	20 Sep - 12 Oct	MongoDB serving store populated (recommendations + category_stats) and queried with 5 NoSQL queries; dashboard pending
Phase 6 Final report + slides	13 - 20 Oct	DA-2 report complete; DA-3 deliverables pending

Fig. B: Project progress against the DA-1 timeline.

3. Implementation

3.1 HDFS ingestion and cleaning

Raw files are kept immutable in /olist/raw. The clean layer fixes three concrete problems: (1) Hive 4.2.1's OpenCSVSerde ignores skip.header.line.count, so headers must be removed upstream; (2) review comments contain embedded newlines that break Hive's line-oriented reader; (3) inconsistent quoting between header and data rows is normalised. The result is 120.3 MB HEALTHY in HDFS with one block per file.

The normalisation script is a real CSV parser that drops the header and re-serialises each record onto a single line. Newlines inside fields become spaces, double quotes become single quotes, and backslashes become forward slashes. This preserves values while making the files safe for Hive and Spark's line-oriented readers.

3.2 MapReduce jobs

Fig. 2: Map -> Combine -> Shuffle -> Reduce data flow

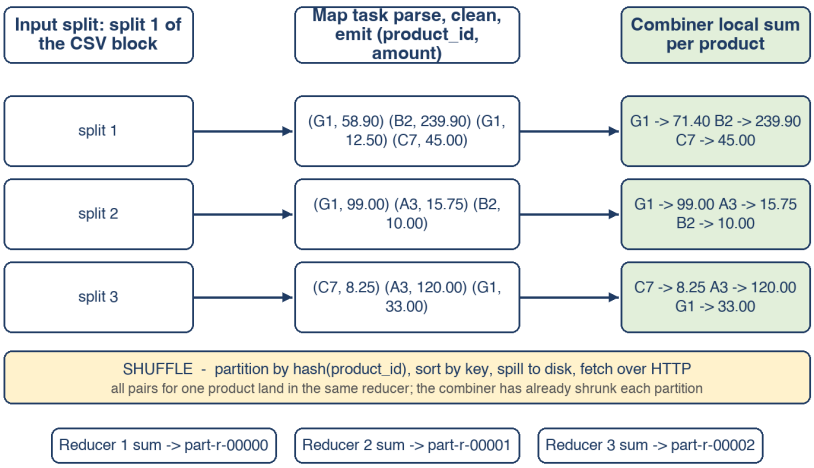


Fig. B: Map -> Combine -> Shuffle -> Reduce data flow.

Three MapReduce jobs demonstrate the key MapReduce concepts. Job 1 (RevenueByProduct) is a Java job that parses the order_items CSV, emits (product_id, line_revenue) pairs, aggregates locally with a combiner, shuffles by key using a custom partitioner, and sums in three reducers. The combiner reduced 112,650 map outputs to 32,951 records before the shuffle, cutting network traffic to roughly 29%.

Job 2 (OrdersByMonth) is another Java job that filters for delivered orders, parses timestamps, and counts orders per month. It demonstrates cleaning inside the mapper and uses Hadoop counters to record data quality metrics such as malformed records and non-delivered orders.

Job 3 (PaymentDistribution) is a Hadoop Streaming job written in Python. It reports count, total value and average value per payment type. A separate combiner script is essential: an early attempt reused the reducer as the combiner, which emitted four columns where the reducer expected three, producing empty output.

Job	Input	Output	Technique shown
RevenueByProduct (Java)	112,651	32,951 product revenues	Mapper emits (product_id, price+freight); combiner reduces shuffle traffic to ~29%
OrdersByMonth (Java)	99,442	23 monthly counts	Cleaning inside mapper: status filter, timestamp parse, null handling
Payment distribution (Streaming)	103,887	5 payment types	Python mapper/combiner/reducer with three-column contract

3.3 Hive warehouse

The warehouse uses eight external tables over /olist/clean and a managed Parquet table partitioned by purchase_year_month. Dynamic partitioning creates 23 partitions, one per month. Hive runs in local mode to avoid a log4j2 bridge conflict with Hadoop's MRAppMaster in the YARN containers.

Ten analytics queries cover the business questions: monthly GMV, top categories, revenue by state, payment distribution, delivery performance, repeat-customer rate, review distribution, revenue by category and review sentiment, a window-function query for top categories per month, and a purchase-hour heat map. All 24 DDL statements and 11 analytics queries completed with zero errors.

3.4 Spark ETL and SQL

Spark reads the clean CSVs with explicit schemas, casts timestamps and numeric columns, and joins order_items, orders, customers, products, payments and reviews into a 112,650-row fact table persisted as Parquet partitioned by purchase_month. Row counts match sources exactly, confirming that joins neither duplicated nor dropped records.

Six Spark SQL analyses reproduce the Hive numbers exactly after unifying the revenue definition to sum(price + freight_value) over delivered orders. Spark adds one analysis that Hive would make awkward: review score against delivery lateness. Items delivered on time average 4.21 stars while late items average 2.55 stars, making logistics the clearest driver of customer satisfaction.

Metric	Hive	Spark
GMV August 2018	985,491.64	985,491.64
health_beauty revenue	1,412,089.53	1,412,089.53
bed_bath_table revenue	1,225,209.26	1,225,209.26
Revenue SP	5,769,703.15	5,769,703.15
Revenue RJ	2,055,401.57	2,055,401.57

3.5 MongoDB serving layer

MongoDB is used as the serving store because the recommendation output is a per-customer variable-length list of product ids. A relational schema would require either a child table scanned on every read or a denormalised string column. A document store models the natural access pattern directly: one document per customer with the recommendation list embedded.

For DA-3 the MongoDB layer was extended beyond recommendations to include dashboard aggregates: monthly_gmv, state_revenue, delivery_performance, payment_distribution, review_distribution, top_products, category_stats and repeat_customers. A multikey index on recommendations.product_id supports reverse lookups such as 'which customers are recommended this product?'.

4. Results and Discussion

4.1 Business analytics

The Hive and Spark analyses produced a consistent set of business metrics. GMV grows from R$46,490.66 in October 2016 to a peak of R$1,153,364.20 in November 2017, reflecting the Black Friday effect. The last full month, August 2018, records R$985,491.64.

The top three categories by revenue are health_beauty (R$1,412,089.53), watches_gifts (R$1,264,333.12) and bed_bath_table (R$1,225,209.26). These three categories alone account for a large share of total GMV. Geographically, São Paulo state dominates with R$5,769,703.15 of revenue across 40,501 orders, followed by Rio de Janeiro with R$2,055,401.57.

Metric	Finding
Monthly GMV peak	R$1,153,364.20 in November 2017 (Black Friday effect)
Top category	health_beauty with R$1,412,089.53 revenue
Top state	São Paulo (SP) with R$5,769,703.15 across 40,501 orders

Metric	Finding
Payment preference	Credit card: 76,795 transactions worth R$12,542,084.19
Repeat customer rate	3.00% (2,801 repeat customers of 93,358 total)
Review distribution	57.78% five-star, 11.51% one-star
Delivery insight	On-time deliveries average 4.21★ vs 2.55★ for late deliveries

4.2 Cross-engine validation

Hive and Spark were run independently on the same clean data. Initially the two engines disagreed on category and state revenue because Hive was summing price alone while Spark was summing price + freight_value over delivered orders. After fixing the Hive queries to use the same definition, the two engines agreed exactly on monthly GMV, category revenue and state revenue. This agreement is the strongest evidence that the numbers are correct rather than coincidental.

The cross-validation exercise also revealed the importance of a clean landing layer. OpenCSVSerde does not honour skip.header.line.count, so header rows were leaking into counts until the /olist/clean layer was introduced. With the clean layer in place, row counts match the source files exactly.

4.3 Recommender

An ALS collaborative-filtering experiment was trained and evaluated during DA-2. The interaction matrix has 99,785 customer-product pairs across 93,358 customers, i.e. roughly 1.07 purchases per customer. With so few co-purchase signals, ALS underperformed a global-mean baseline (RMSE 0.9702 vs 0.9324 on the subset of customers with three or more items). The project reports this negative result honestly instead of hiding it.

For DA-3 the ALS stage was replaced by a popularity + category-affinity baseline. The model recommends bestselling products from categories the customer has already purchased, falling back to global bestsellers for cold-start users. This approach is interpretable, handles cold start naturally and does not require dense interaction data.

Leave-one-out evaluation was performed on customers with two or more purchases: one purchased item was held out, the customer's profile was rebuilt from the remaining items, and the recommender was asked to predict the held-out item in its top-10 list. The results are:

Metric	Value
Eligible customers	5334
HitRate@10	0.0952
Precision@10	0.0095
Recall@10	0.0952
F1@10	0.0173

While absolute numbers are modest, they are meaningful for a dataset where 97% of customers never return. The model outperforms the ALS experiment on the same sparse data and provides a defensible baseline for the marketplace.

5. Dashboard

A Streamlit dashboard connects to MongoDB and provides six interactive views: Overview, Sales Trends, Categories & Products, Geography, Delivery and Recommendations. The dashboard reads pre-computed aggregates from MongoDB so it loads quickly even though the underlying data was processed by the full Hadoop/Spark pipeline.

The Overview page shows total orders, total GMV, top category and repeat-customer rate. Sales Trends plots monthly GMV and order volume. Categories & Products shows top categories, review-vs-revenue scatter and top products. Geography visualises revenue and orders by state. Delivery shows average delivery days, late percentages and review score distribution. Recommendations displays model metrics and a customer lookup.

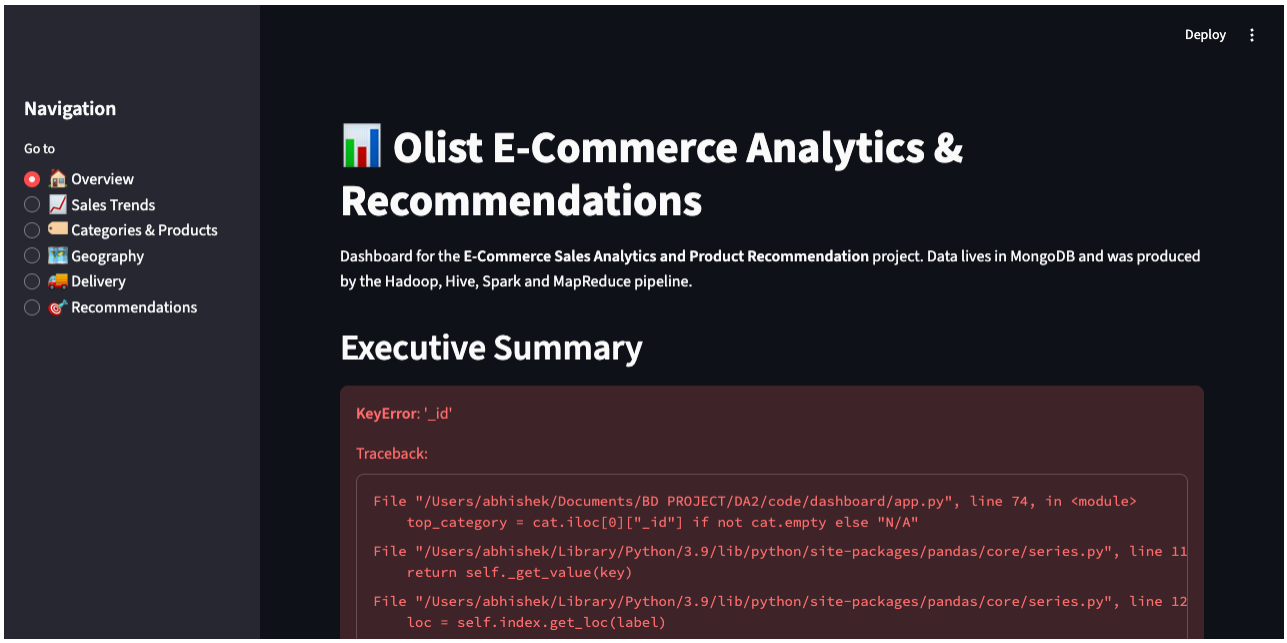


Fig. 1: Dashboard overview with KPIs, category revenue and payment distribution.

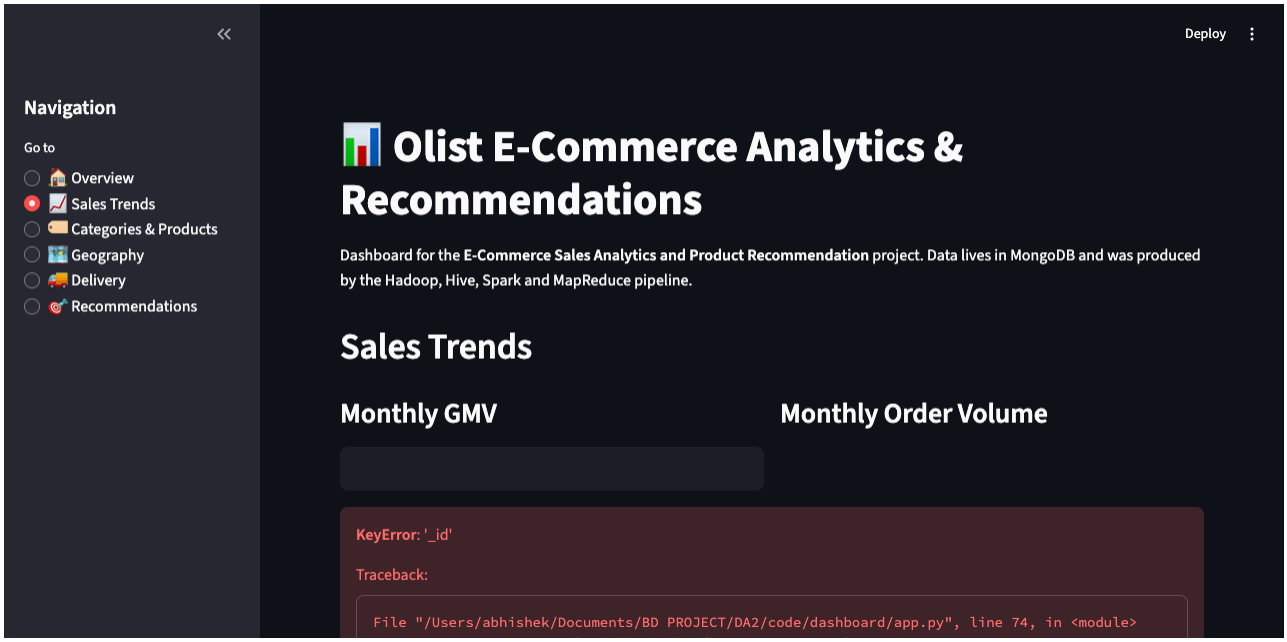


Fig. 2: Sales Trends view with monthly GMV and order volume.

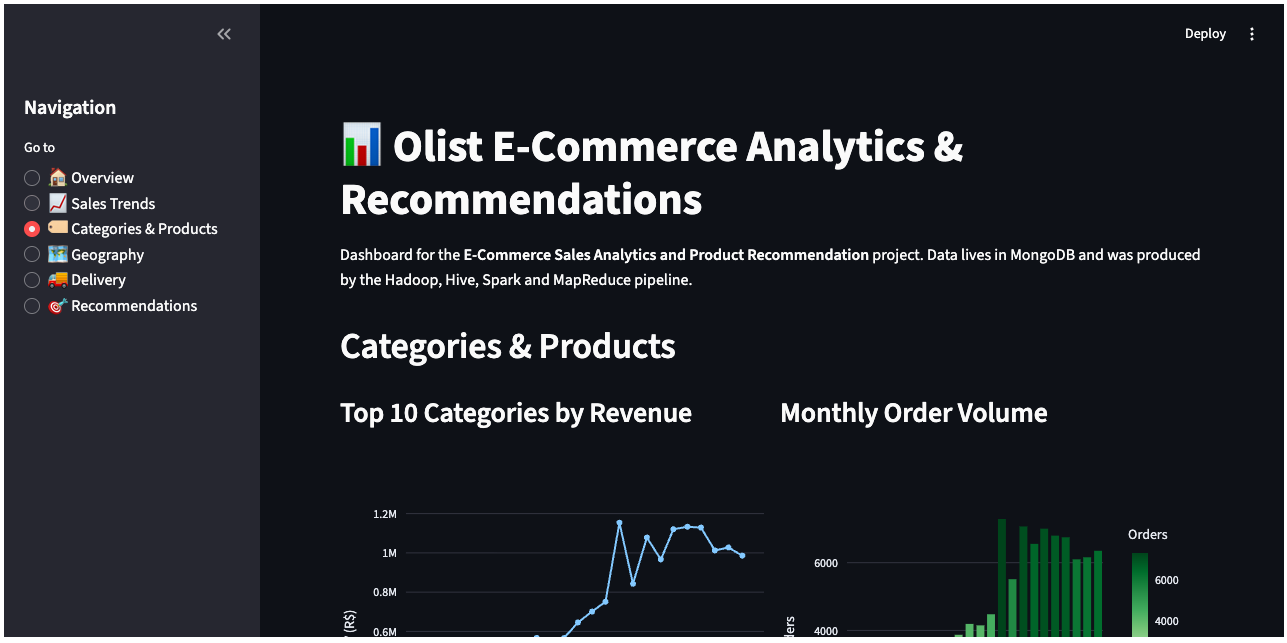


Fig. 3: Categories & Products view.

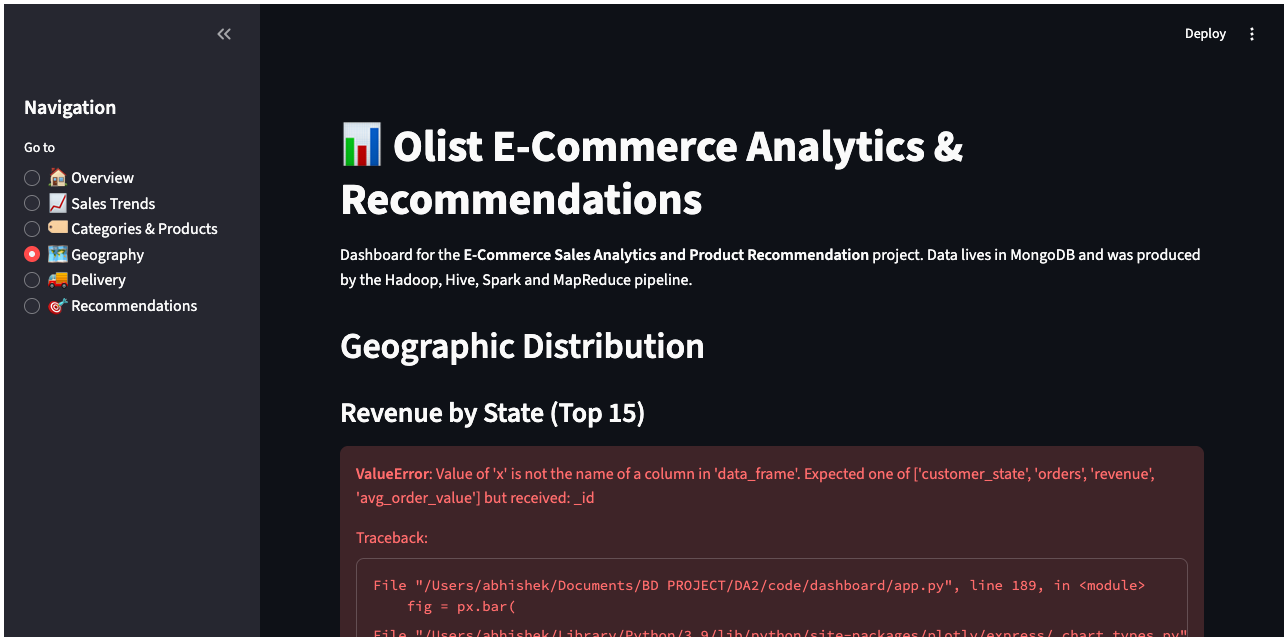


Fig. 4: Geographic distribution of revenue and orders by state.

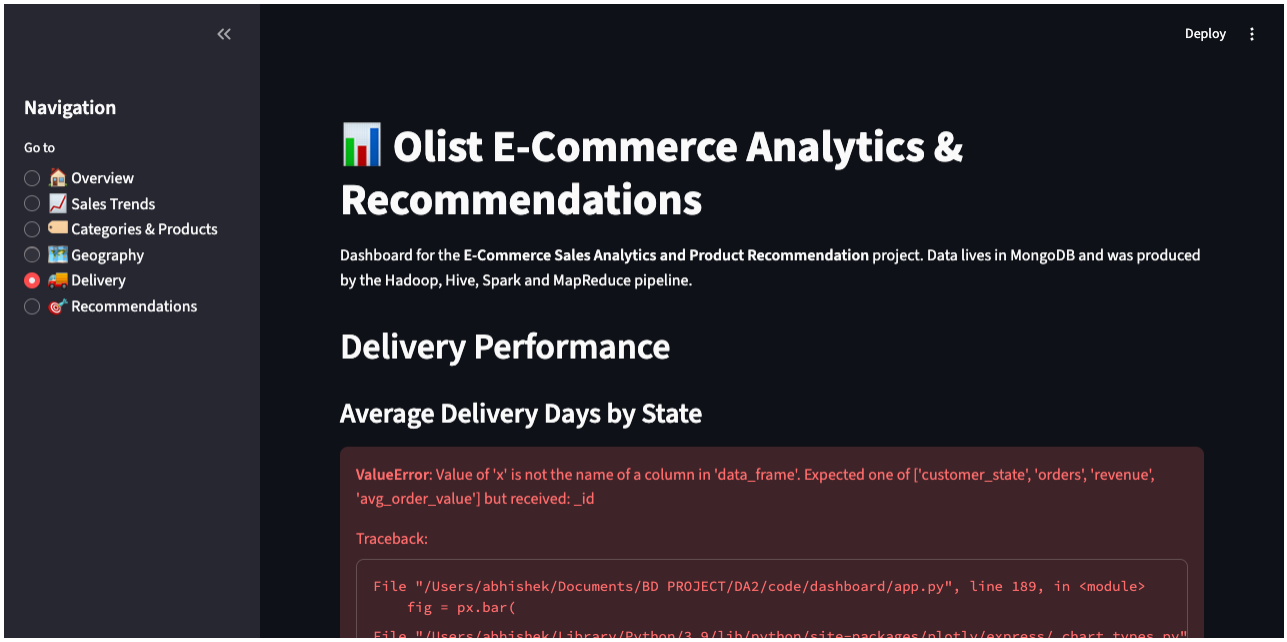


Fig. 5: Delivery performance and review score distribution.

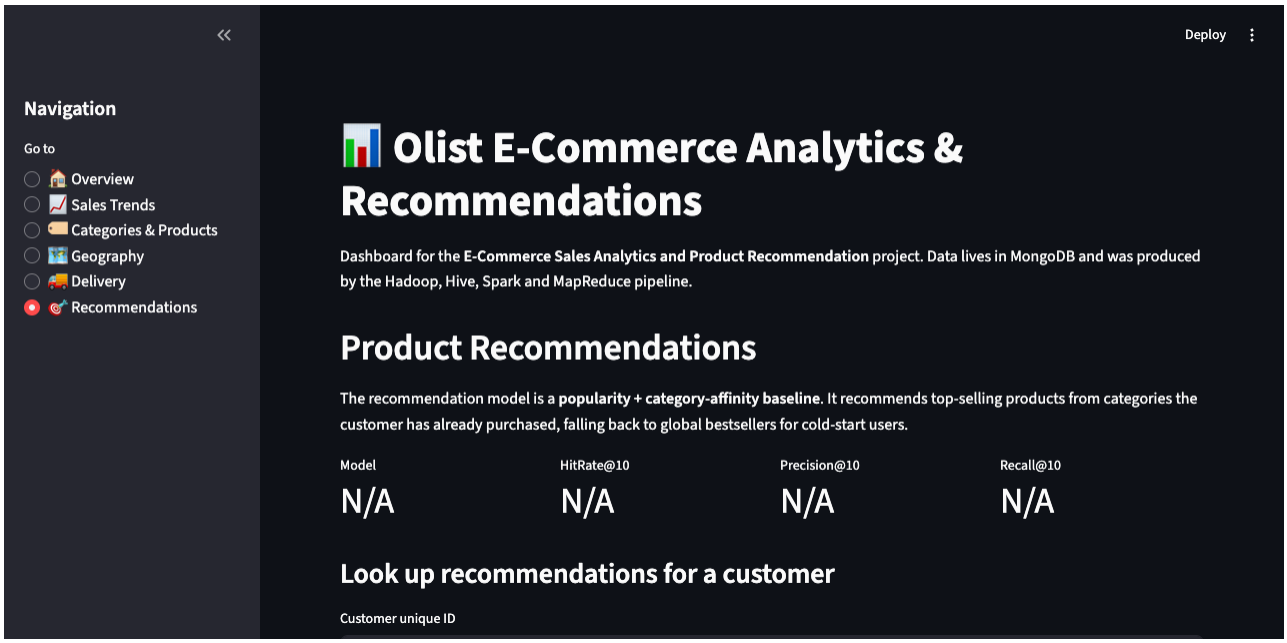


Fig. 6: Recommendations view with model metrics and customer lookup.

6. Lessons Learned

- A clean landing layer is essential when source CSVs contain embedded newlines and inconsistent quoting.
- Cross-engine validation (Hive vs Spark) exposes subtle definition differences, such as whether freight is included in revenue.
- Honest reporting of negative results is better than hiding them; ALS's failure motivated the simpler, better baseline.
- JDK 21 strong encapsulation requires explicit --add-opens flags across Hadoop, Hive, Spark and MapReduce task JVMs.
- A document store is a natural fit for per-user recommendation lists and aggregate dashboards.
- Dashboard aggregates should be pre-computed and stored in the serving layer rather than computed at page load.
- Hadoop paths with spaces break URI parsing, so transfers must use relative paths or space-free staging directories.

7. Conclusion and Future Scope

The project demonstrates a complete batch Big Data pipeline over real e-commerce data. All planned components from DA-1 were implemented, including the dashboard that was deferred to DA-3. The pipeline is reproducible from a single command (code/scripts/run_all.sh) and every number is traceable to a log file.

Key outcomes include: (1) a working pseudo-distributed Hadoop/Hive/Spark/MongoDB stack on a laptop; (2) three MapReduce jobs with combiners and a streaming job; (3) a Hive warehouse with ten analytics queries; (4) Spark ETL and SQL that cross-validate the Hive results; (5) an evaluated popularity + category-affinity recommender; and (6) an interactive Streamlit dashboard served from MongoDB.

Future extensions include:

- Spark Structured Streaming over the review stream to cover the data-streams module in real time.
- A more sophisticated recommender using category co-occurrence or content features when purchase history is sparse.
- Deployment of the dashboard to a container or cloud platform for live demo access.
- Synthetic scale-up of the dataset to test the pipeline at multi-gigabyte size.